

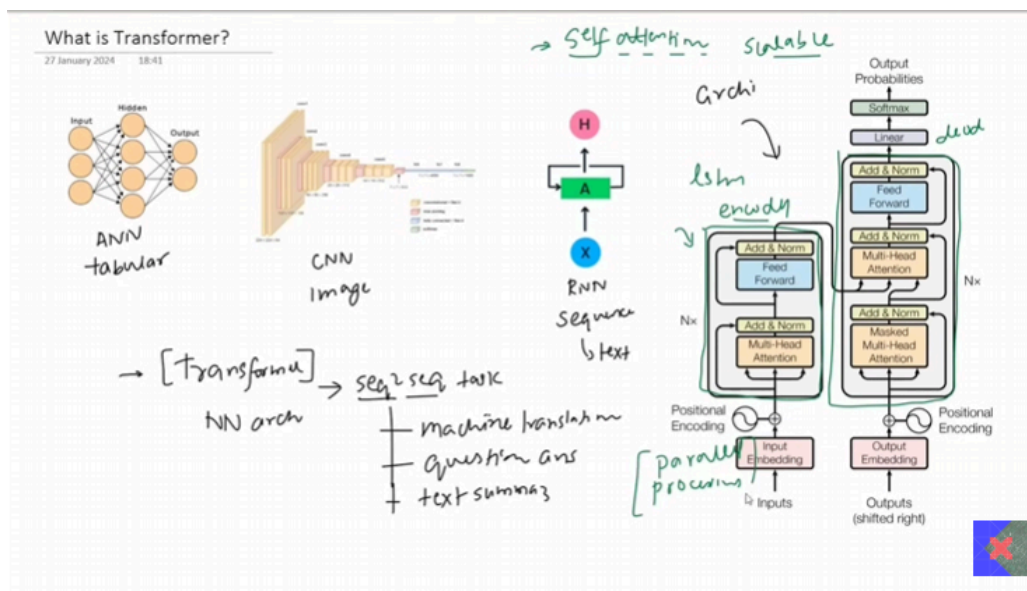
Transformers Part 1 - Basic Understanding

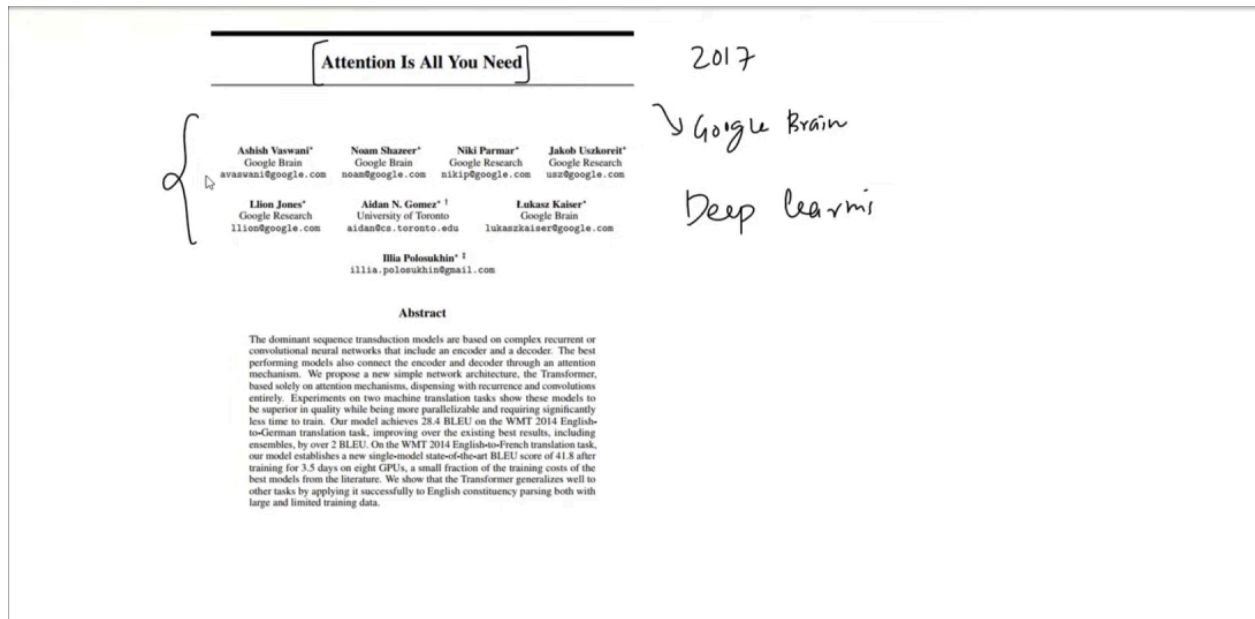
Introduction to Neural Networks & What is a Transformer?

Data Types and Traditional Architectures

Different deep learning architectures were traditionally built for specific data modalities:

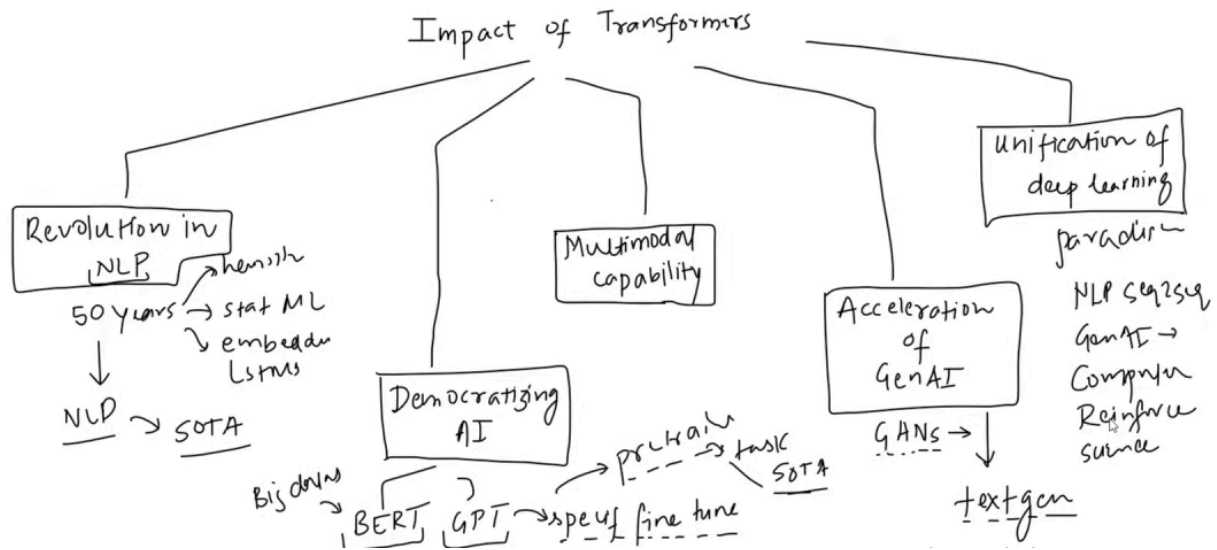
- ANN (Artificial Neural Networks): Best suited for structured, tabular data (e.g., spreadsheets, CSV files).
- CNN (Convolutional Neural Networks): Designed for spatial grid data, primarily images and computer vision.
- RNN (Recurrent Neural Networks): Designed for sequential and time-series data, specifically natural language and text.





A Transformer is a neural network architecture introduced in 2017 by Google researchers in the paper "*Attention Is All You Need*".

- Designed primarily for Sequence-to-Sequence (Seq2Seq) tasks where an input sequence is converted into an output sequence:
 - Machine Translation (e.g., English to French)
 - Question Answering
 - Text Summarization
- The Core Breakthrough: Unlike RNNs, which read text one word at a time sequentially, Transformers use Self-Attention and Parallel Processing. They process all words in a sentence at the same time, making them vastly more scalable and faster to train on modern hardware (GPUs).



1. **Revolution in NLP:** Ended a 50-year evolution from rule-based and statistical NLP to word embeddings (Word2Vec) and LSTMs, establishing a new state-of-the-art (SOTA).
2. **Democratization of AI:** Introduced the **Pre-train $\rightarrow$ Fine-tune** paradigm. Tech giants pre-train massive foundation models (like BERT or GPT) on vast datasets; practitioners can fine-tune them on small, task-specific datasets with minimal compute.
3. **Multimodal Capability:** The same core architecture that processes words can now process image patches, audio waveforms, video frames, and sensor readings.
4. **Acceleration of Generative AI:** Moved beyond GANs (Generative Adversarial Networks) to robust, autoregressive text generation (ChatGPT, Claude) and multimodal generation.
5. **Unification of Deep Learning Paradigms:** Provided a single common backbone across NLP, Computer Vision (ViTs), Reinforcement Learning (Decision Transformers), and Biology (AlphaFold).

The Origin Story!

27 January 2024 22:38

Sequence to Sequence Learning with Neural Networks

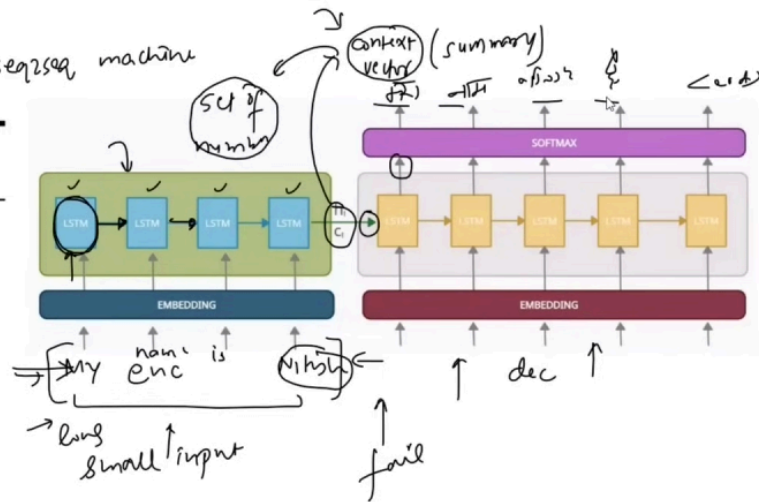
Ilya Sutskever
Google
ilyasu@google.com

Oriol Vinyals
Google
vinyals@google.com

Quoc V. Le
Google
qvl@google.com

Abstract

Deep Neural Networks (DNNs) are powerful models that have achieved excellent performance on difficult learning tasks. Although DNNs work well whenever large labeled training sets are available, they cannot be used to map sequences to sequences. In this paper, we present a general end-to-end approach to sequence learning that makes minimal assumptions on the sequence structure. Our method uses a multilayered Long Short-Term Memory (LSTM) to map the input sequence to a vector of a fixed dimensionality, and then another deep LSTM to decode the target sequence from the vector. Our main result is that on an English to French translation task from the WMT'14 dataset, the translations produced by the LSTM achieve a BLEU score of 34.8 on the entire test set, where the LSTM's BLEU score was penalized on out-of-vocabulary words. Additionally, the LSTM did not have difficulty on long sentences. For comparison, a phrase-based SMT system achieves a BLEU score of 33.3 on the same dataset. When we used the LSTM to rerank the 1000 hypotheses produced by the aforementioned SMT system, its BLEU score increases to 36.5, which is close to the previous best result on this task. The LSTM also learned sensible phrase and sentence representations that are sensitive to word order and are relatively invariant to the active and the passive voice. Finally, we found that reversing the order of the words in all source sentences (but not target sentences) improved the LSTM's performance markedly, because doing so introduced many short term dependencies between the source and the target sentence which made the optimization problem easier.



The Origin Story: The Bottleneck of Seq2Seq (2014–2015)

Before Transformers, sequence-to-sequence tasks relied on recurrent networks:

- **Sutskever et al. (2014) – Sequence to Sequence Learning with Neural Networks:**
 - Used an **Encoder LSTM** to compress the entire input sentence into a single, fixed-size vector (the "context vector" or "thought vector").
 - A **Decoder LSTM** then generated the target sentence from this single vector.
 - **The Core Flaw (Information Bottleneck):** Trying to squeeze a 50-word sentence into a fixed-size vector caused the network to "forget" the beginning of the sentence. It worked well for short inputs, but translation quality degraded drastically on longer sentences (>30 words).

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dmitry Bahdanau
Jacobs University Bremen, Germany
KyungHyun Cho Yoshua Bengio*
Université de Montréal

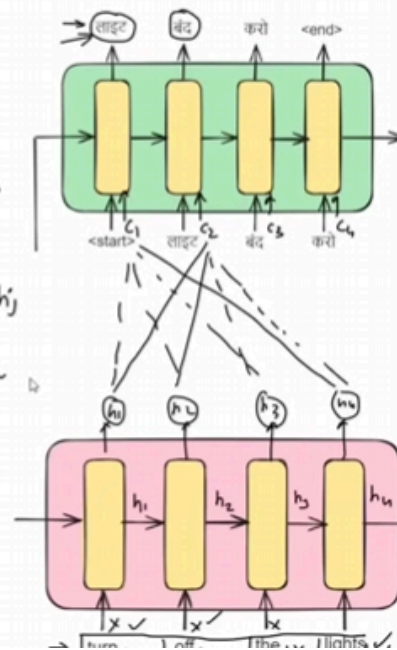
ABSTRACT

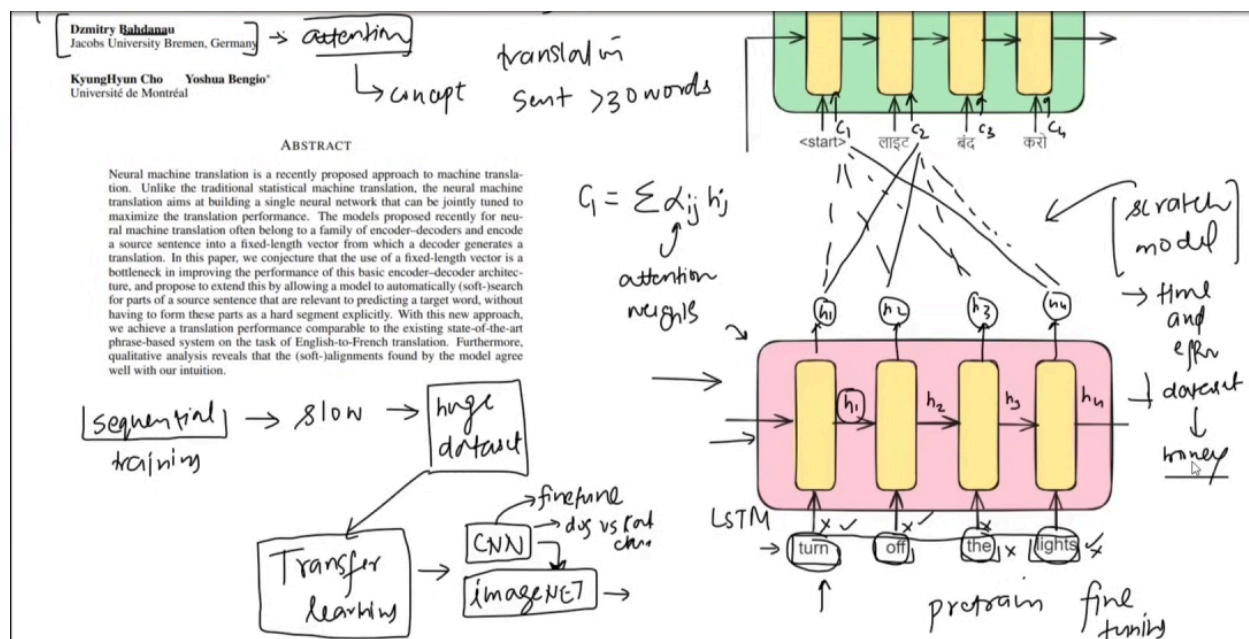
Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly trained to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder-decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.

translation
sent > 30 words

$$q_i = \sum_j d_{ij} h_j$$

attention weights





The First Breakthrough: Bahdanau Attention (2014–2015)

In *Neural Machine Translation by Jointly Learning to Align and Translate*, Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio solved the fixed-vector bottleneck.

The Concept

Instead of compressing the whole sentence into a single static vector, the decoder is allowed to "look back" at all the encoder's hidden states at every step and focus on the most relevant words.

Mathematical Formulation

The dynamic context vector C_i for decoding step i is a weighted sum of all encoder hidden states h_j :

$$C_i = \sum_j \alpha_{ij} h_j$$

Where:

- h_j is the hidden state of the j -th input word from the encoder.
- α_{ij} is the **attention weight** (a score between 0 and 1) showing how much the i -th target word should focus on the j -th source word.

Example from the Notes

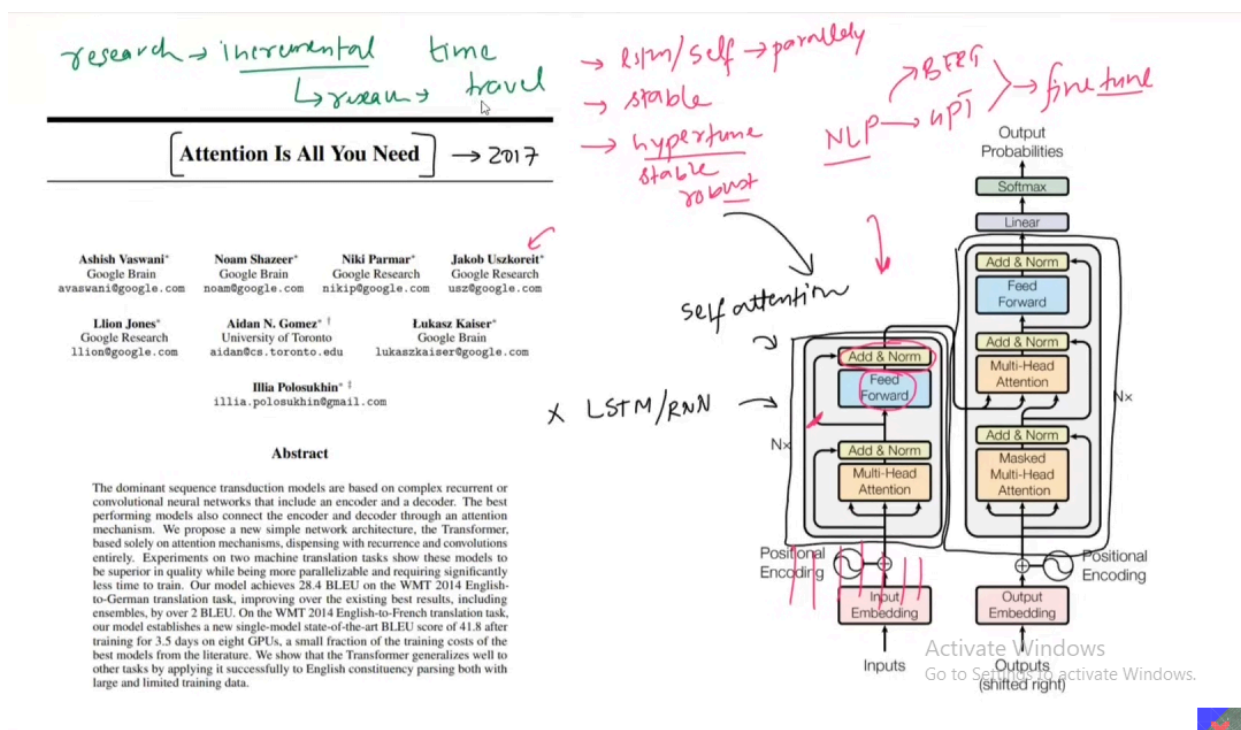
Translating: "turn off the lights"-----> "लाइट बंद करो"

- When generating "लाइट", the attention weights put high focus on the encoder state for "lights".
- When generating "बंद करो", attention shifts to "turn off".

Remaining Limitations

Even with Bahdanau attention, the underlying engine was still an **LSTM/RNN**:

- **Sequential processing:** Step t cannot run until step t-1 finishes. Training was fundamentally slow.
- Models had to be trained from scratch with massive datasets, costing significant time, energy, and money.
- Computer Vision already had **Transfer Learning** (pre-training on ImageNet, fine-tuning on cats vs. dogs), but NLP still lacked an efficient parallel backbone to do the same.



Google researchers (Vaswani et al.) posed a bold question: *What if we remove LSTMs/RNNs entirely and rely only on Attention?*

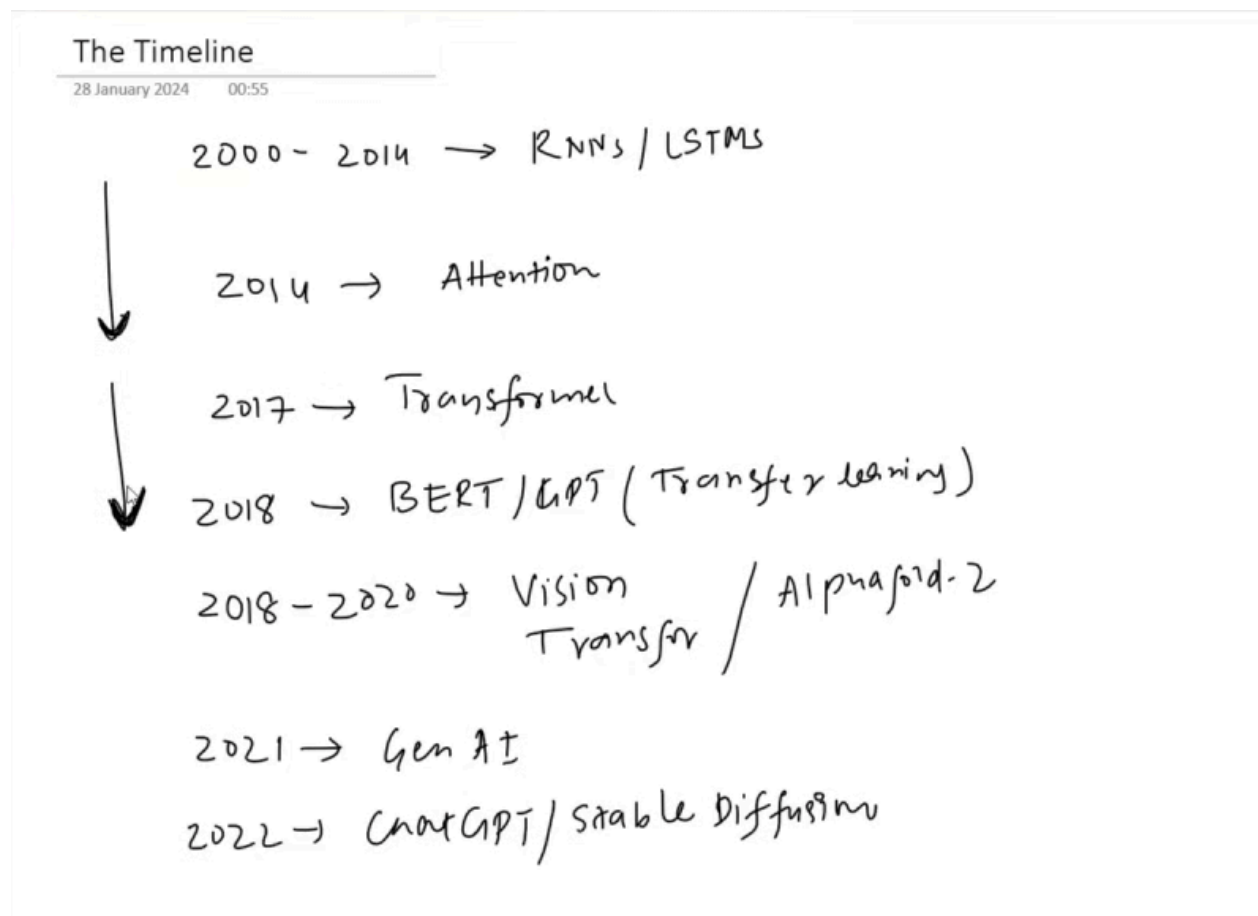
Self-Attention: Computes relationships between every pair of words in the sentence simultaneously.

Positional Encoding: Because recurrence was removed and words enter simultaneously, the model has no inherent sense of word order. Positional vectors are added directly to the input embeddings to inject sequential order.

Multi-Head Attention: Multiple attention mechanisms run in parallel, allowing the model to focus on different aspects at the same time (e.g., grammatical structure, pronouns, semantic meaning).

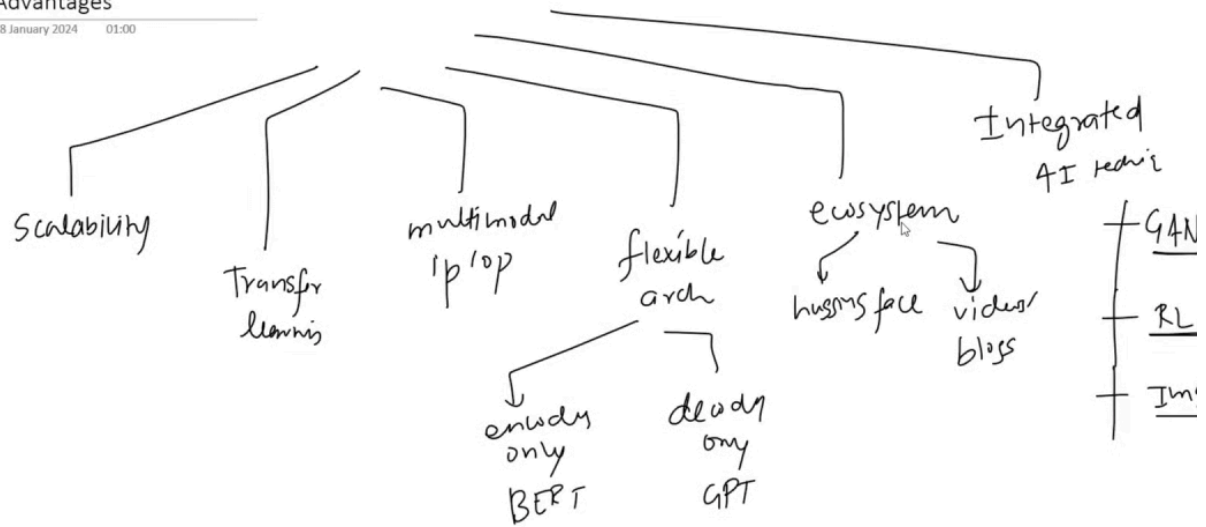
Masked Multi-Head Attention (in Decoder): Prevents the decoder from "cheating" by looking ahead at future tokens during training; it only attends to earlier positions.

Residual Connections & LayerNorm (Add & Norm): Stabilizes training and prevents vanishing gradients, enabling deep stacking of layers (N X).



Advantages

28 January 2024 01:00

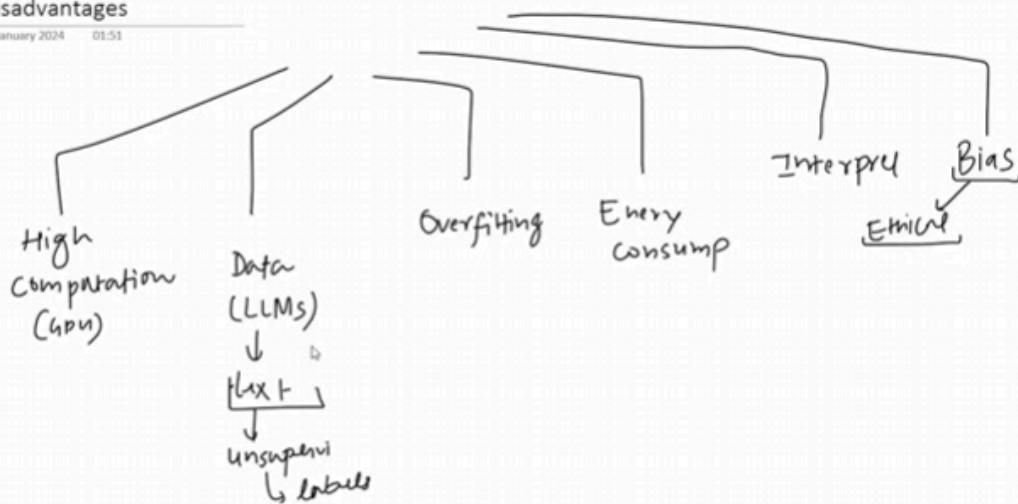


Activate Windows
Go to Settings to activate Windows.



Disadvantages

28 January 2024 01:51



Future

28 January 2024 02:09

